MediCore · Matriz inicial de trazabilidad

Estados: ✓ hecho y verificado · ◐ parcial · ○ pendiente. "Implementación" y "Pruebas" apuntan a rutas reales cuando existan; hoy solo hay lo de la entrega 0.

Entregables de pruebas obligatorios

Las 304 pruebas xUnit (48 unitarias, 50 de la lógica móvil y 206 de integración con PostgreSQL, Redis y RabbitMQ reales; ninguna omitida) son complementarias. El documento exige además estos entregables:

Entregable (documento, secciones 4, 8 y 12)	ID	Qué exige	Estado	Entrega
Cypress (web)	GEN-10	Smoke, flujo funcional crítico, regresión y al menos un escenario negativo	✓ **EJECUTADO** — 11 specs, 116/116 en Chrome contra QA (102 funcionales y 14 de accesibilidad). Electron y dispositivo físico sin repetir	10
Insomnia (API)	GEN-17	Colección con pruebas positivas, negativas, de autenticación, autorización y errores	✓ **EJECUTADO** — 61 solicitudes, 65/65 comprobaciones contra QA	10
Maestro (móvil)	GEN-11	Entre 5 y 8 flujos de la app, con al menos un escenario negativo	✓ **EJECUTADO** — `tests/maestro`: 8 flujos (login válido e inválido, reserva, consulta y cancelación, resultados, recetas, notificaciones, cierre de sesión; con negativos) **8/8 aprobados con el código final en emulador Android 35, contra QA** (`documentacion/evidencias/maestro-codigo-final.md`). Límite: sin dispositivo físico	7, 10
Rendimiento con k6 o JMeter	GEN-12, CRI-01, CRI-02	Load, Stress y Spike con umbrales declarados antes de ejecutar	✓ **EJECUTADO** — umbrales cumplidos, 0 solapes y 0 pendientes; medición local (`07-rendimiento.md`)	13

Cypress cubre ahora los recorridos web de Acceso, Recepción, Administrador, Médico y Laboratorio (ver fila GEN-10); **no cubre** el smoke del pipeline ni una auditoría de accesibilidad automatizada; el caso "RabbitMQ detenido" no es automatizable desde Cypress (se cubre con `MessagingTests` y `scripts/evidencia_laboratorio.py`) y la caída de la API se comprobó con `scripts/web_conexion.py`.

Lo que sí existe hoy y se reutilizará: pruebas de concurrencia por API (12 solicitudes al mismo horario, 24 pacientes por 4 horarios con lecturas simultáneas), que sirven como base del escenario pero **no cuentan como prueba de rendimiento** (no miden p95 ni throughput y no usan umbrales previos). La verificación manual en navegador está descrita en `04-registro-de-hallazgos.md` y tampoco sustituye a Cypress.

Funcionales

ID	Requisito (fuente)	Pantallas	Implementación	Pruebas previstas	Entrega	Estado
RF-01	Gestión de pacientes y médicos (P2)	WEB-05…08	`PatientsController`, `DoctorsController`, `Pages/Pacientes`, `Pages/Medicos`	`PatientsApiTests`, `DoctorsApiTests`, `PatientRulesTests`; Cypress pendiente	3	◐
RF-02	Agenda y reserva de citas (P2)	WEB-02, 03, 04, 09, 11; MOB-03…07 (app: `Pages/Appointments*`, `Doctors`, `Slots`, `Confirm`)	`AppointmentsController`, `DoctorSchedulesController`, `Pages/Agenda`, `Pages/MiAgenda`, `Pages/Medicos/Disponibilidad`	`AppointmentsApiTests`, `AvailabilityApiTests`, `SchedulingRulesTests`; Cypress: recorrido de Recepción (`recepcion-reserva.cy.js`, 8 pruebas); Maestro `tests/maestro`	4	✓
RF-03	Consultas médicas (P2)	WEB-12, 13	`ClinicalController` (consultas), `Pages/Pacientes/Expediente`, `RegistrarConsulta`; entidad `Consultation` (una por cita, con paciente, médico y cita)	`ClinicalApiTests`, `ResourceAuthorizationTests`; Cypress `medico-expediente.cy.js`	5	◐ (la lectura por el paciente no está en el diseño)
RF-04	Recetas (P2)	WEB-14, 15; MOB-10, 11 (app: `Pages/Prescription*`)	`ClinicalController` (recetas con 1 a 10 medicamentos), `Pages/Pacientes/CrearReceta`, `Receta`; `GET /api/prescriptions/me` para el móvil	`ClinicalApiTests`, `ResourceAuthorizationTests`; Cypress `medico-expediente.cy.js`; Maestro `tests/maestro`	5, 9	✓
RF-05	Registro/consulta de resultados de laboratorio (P2)	WEB-16…18; MOB-08, 09 (app: `Pages/LabResult*`)	`LabsController`, `LabResultAuthorizationHandler`, `Pages/Laboratorio`, pestaña Laboratorio de `Pages/Pacientes/Expediente`	`LaboratoryApiTests` (registro, validación, roles, IDOR, atomicidad, reintento, auditoría sin contenido), `LabResultAuthorizationTests`, `LabResultRequestValidationTests`; Cypress `laboratorio-resultados.cy.js`; Insomnia y Maestro pendientes	6	✓ (web y API; móvil pendiente)
RF-06	Notificaciones al paciente (P2)	WEB-18; MOB-12 (app: `Pages/Notifications Page`)	`NotificationsController` (`GET /api/notifications`, `unread-count`, `POST /{id}/read`), `Notification`, `NotificationConsumerService`	`LaboratoryApiTests` (notificaciones propias y ajenas), `MessagingTests` (notificación asíncrona real); Maestro `tests/maestro`	6	✓
RF-07	Aplicación móvil para pacientes (P2)	MOB-01…12	`src/MediCore.Mobile.Core` (cliente, sesión, ViewModels), `src/MediCore.Mobile` (vistas MAUI)	`MediCore.Mobile.Tests` (50), `MobileApiIntegrationTests` (17, API real); **Maestro 8/8 en emulador Android**	7	✓ (8/8 en emulador; sin dispositivo físico)

ID	Requisito (fuente)	Pantallas	Implementación	Pruebas previstas	Entrega	Estado
RF-08	Usuarios y roles (diseño)	WEB-10	`UsersController`, `Pages/Usuarios.cshtml`	`IdentityApiTests` (RBAC, cambio de rol); Cypress `administrador.cy.js` (cambio de rol y su efecto, cuenta propia, acceso por rol)	2	✓

API de referencia

ID	Endpoint	Implementación	Pruebas previstas	Entrega	Estado
API-00	`GET /api/health` y `/api/health/ready` (técnico)	`src/MediCore.Api/Controllers/HealthController.cs`	`BasicEndpointTests`, `PostgresIntegrationTests` (API↔PostgreSQL)	0–1	✓
API-A1	`POST /api/auth/login`, `/refresh`, `/logout`, `GET /api/auth/me` (base de MOB-01)	`AuthController`	`IdentityApiTests`	2	✓
API-A2	`GET /api/users`, `PUT /api/users/{id}/role`, `GET /api/audit` (administrador)	`UsersController`, `AuditController`	`IdentityApiTests`	2	✓
API-01	`GET /api/doctors` (+ `GET/POST/PUT /api/doctors/{id}`, administrador)	`DoctorsController`	`DoctorsApiTests`; Insomnia pendiente	3	✓
API-02	`GET /api/doctors/{id}/availability?date=` (+ `GET/PUT /api/doctors/{id}/schedule`)	`DoctorSchedulesController`, `AvailabilityService`	`AvailabilityApiTests`; Insomnia pendiente	4	✓
API-03	`POST /api/appointments` (+ `GET` agenda, `GET /{id}`, `GET /me`, `POST /{id}/cancel`) con `Idempotency-Key`	`AppointmentsController`	`AppointmentsApiTests` (concurrencia, idempotencia, cancelación, autorización, datos inválidos); Insomnia pendiente	4	✓
API-04	`GET /api/patients/{id}` (+ listado/búsqueda, `me`, alta y edición)	`PatientsController`, `PatientAuthorizationHandler`	`PatientsApiTests` (IDOR, roles, duplicados, concurrencia); Insomnia pendiente	3	✓
API-05	`POST /api/labs/results` (+ listado, detalle, `labs/patients`, `me`, `patients/{id}/lab-results`, reintento de notificación)	`LabsController`, `LabResultAuthorizationHandler`	`LaboratoryApiTests` (43 casos), `MessagingTests` (caída de RabbitMQ); evidencia `documentacion/evidencias/entrega-6-laboratorio.md`; Insomnia pendiente	6	✓
API-06	`GET /api/prescriptions/{id}` (+ crear, listar por paciente y `me`) y consultas	`ClinicalController`, `PrescriptionAuthorizationHandler`	`ClinicalApiTests` (autor y paciente dueño; terceros 404; roles 403); Insomnia pendiente	5	✓

Redis y RabbitMQ

ID	Requisito	Implementación	Pruebas previstas	Entrega	Estado
RED-01	Cache de disponibilidad médica (TTL, invalidación al reservar/cancelar o cambiar el horario)	`Infrastructure/Scheduling/AvailabilityService.cs`, `CacheKeys.Availability`, `RemoveByPrefixAsync`	`AvailabilityApiTests`: MISS, HIT, TTL, invalidación, prefijo por médico	4	✓
RED-02	Cache de catálogos y consultas frecuentes	Catálogo de médicos: `Infrastructure/Caching/CacheService.cs`, TTL 5 min, invalida al crear/editar	`DoctorsApiTests`: MISS, HIT, TTL, invalidación	3–4	◐ (falta disponibilidad: entrega 4)
RED-03	Sesiones en Redis	`Infrastructure/Identity/SessionStore.cs`	`IdentityApiTests`: logout, refresh, reutilización, Redis caído	2	✓
RED-04	Recuperación si Redis no está disponible (degradar a BD)	El cache falla abierto y consulta PostgreSQL (catálogo y disponibilidad); las sesiones fallan cerrado (RED-03)	`DoctorsApiTests` y `AvailabilityApiTests` (consulta y reserva con Redis inalcanzable), `IdentityApiTests`	3–4	✓
MQ-01	Eventos AppointmentCreated / AppointmentCancelled	`AppointmentsController` → outbox en la misma transacción → `PatientEventHandler`	`PatientEventsTests` (RabbitMQ real: reserva, cancelación concurrente, broker detenido, duplicado)	7	✓
MQ-02	LabResultReady, PrescriptionCreated, PatientNotification	Los tres por outbox; cada notificación creada emite `PatientNotification` en su misma transacción y su consumo marca `DispatchedAtUtc`	`MessagingTests`, `PatientEventsTests`	6–7	✓
MQ-03	Consumidor con reintentos, DLQ e idempotencia	`NotificationConsumerService`, `LabResultReadyHandler`, topología en `Messaging/Topology.cs` (reintento con espera por mensaje, DLQ), unicidad en BD de `Notification`	`MessagingTests`: fallo dos veces y luego procesa, fallo permanente → DLQ y recuperación con "reintentar", mensaje ilegible o inexistente → DLQ directa, mensaje reentregado y evento republicado sin duplicar	6–7	✓ (para `LabResultReady`; se generaliza en la entrega 7)
MQ-04	Caída del broker no pierde datos clínicos (outbox)	Resultado + evento en una sola transacción (`LabsController.Create`); `OutboxPublisherService` con confirmaciones, `FOR UPDATE SKIP LOCKED` y espera creciente; migración `LaboratorioYNotificaciones`	`MessagingTests.Con_el_broker_detenido…` (Testcontainers) y `scripts/evidencia_laboratorio.py` (contra el RabbitMQ de compose: detener, registrar, recuperar), `LaboratoryApiTests.Resultado_y_evento_se_guardan_juntos_o_ninguno`	6	✓

Escenario crítico y seguridad

ID	Requisito	Implementación	Pruebas previstas	Entrega	Estado
CRI-01	Sin doble reserva ni colisión de horario bajo alta demanda	Restricciones de exclusión en PostgreSQL (médico y paciente) + candados de asesoría por horario + reintento ante deadlock	`AppointmentsApiTests`: 12 solicitudes al mismo horario → 1 reserva; 24 pacientes por 4 horarios + lecturas; mismo paciente con 6 médicos; reenvíos simultáneos con la misma clave. Carga con k6 pendiente (entrega 13)	4, 13	◐
CRI-02	Consistencia con lecturas concurrentes de agenda/expediente	`tests/k6/medicore.js`, `scripts/perf-integridad.sh`	k6 mixto: 0 errores inesperados y 0 solapes	13	✓
CRI-03	RabbitMQ interrumpido después de registrar un resultado	Ver MQ-04	`MessagingTests.Con_el_broker_detenido…`, evidencia `documentacion/evidencias/entrega-6-laboratorio.md` (sección B)	6	✓
SEC-01	RBAC por rol clínico/administrativo	`[Authorize(Roles)]`, política fallback, `Contracts/Roles.cs`	`IdentityApiTests.Solo_el_administrador…`, `BasicEndpointTests`	2	◐ (roles clínicos con sus módulos)
SEC-02	Autorización a nivel de recurso (IDOR)	Pacientes, citas, recetas, consultas y resultados de laboratorio: `PatientAuthorizationHandler`, `AppointmentAuthorizationHandler`, `PrescriptionAuthorizationHandler`, `LabResultAuthorizationHandler` y `ICareRelationship`. **Laboratorio:** Laboratorio lee todos los resultados, el médico solo con relación válida, el paciente solo los suyos, Recepción y Administrador nunca (403); ajeno → 404 + `clinical.access_denied`. **Expediente compartido:** el médico con relación de atención válida (cita no cancelada o consulta registrada) lee los registros de otros médicos; sin relación, 404; nadie modifica lo de otro (solo el autor crea recetas sobre su consulta; no hay edición ni borrado); 404 y auditoría ante intento ajeno	`PatientsApiTests`, `AppointmentsApiTests`, `ClinicalApiTests` (`Expediente_compartido_*`), `ResourceAuthorizationTests` (unitarias, sin BD); Cypress `medico-expediente.cy.js` (colega)	3–6	◐ (falta laboratorio: entrega 6)

ID	Requisito	Implementación	Pruebas previstas	Entrega	Estado
SEC-03	Exposición de datos sensibles (respuestas, logs, errores)	Información clínica solo para quien corresponde; la auditoría y las bitácoras no contienen motivo, notas, diagnóstico ni medicamentos; el paciente no ve contacto de médicos; ProblemDetails	`ClinicalApiTests.Ni_la_auditoria_ni_las_bitacoras_contienen_el_contenido_clinico` y `LaboratoryApiTests.Ni_la_auditoria_ni_las_bitacoras_contienen_el_contenido_del_resultado` (con control del capturador); el evento del outbox lleva solo identificadores, `DoctorsApiTests`; revisión ZAP pendiente	3–6	◐
SEC-04	Sesiones y autenticación (expiración, bloqueo, logout)	`AuthController`, JWT, `User` (bloqueo)	`IdentityApiTests` (expiración, manipulación, bloqueo, logout), `UserTests`; Cypress negativo pendiente	2	✓
SEC-05	Seguridad de API (validación, rate limit, CORS, cabeceras)	ProblemDetails, validación de entrada, rate limit en login/refresh, CSP y cabeceras en la web	`IdentityApiTests.Login_se_limita_por_ip`; Insomnia y ZAP pendientes	1–2	◐
SEC-06	Trazabilidad y auditoría de accesos y cambios clínicos	`audit_entries`, `IAuditLogger`, `GET /api/audit`	`IdentityApiTests`; pacientes y médicos (`PatientsApiTests`); citas: creadas, reenviadas, conflictos, canceladas, leídas, agenda consultada, accesos denegados, horario cambiado (`AppointmentsApiTests`, `AvailabilityApiTests`)	2–6	◐
SEC-07	Secretos fuera del repositorio	`.gitignore`, `.env.example`	Revisión + escaneo en pipeline	0	◐

Generales / DevSecOps / documentos

ID	Requisito	Implementación	Entrega	Estado
GEN-01	Repo GitHub: ramas, PR, Issues, Releases, protección de `main`	Remoto en GitHub, ramas `feature/*` y PR #1 fusionado; falta protección de `main`, Issues y Releases	0, 11	◐
GEN-02	Jenkinsfile con las 18 etapas + bloqueo por Quality Gate	`Jenkinsfile`, `ci/`, `scripts/quality-gate.py`; 18 etapas ejecutadas (corrida 5) y bloqueo demostrado (corrida 6), `reports/jenkins/`	11	✓
GEN-03	Dockerfiles + docker-compose	`docker/{api,web,worker,migrator}.Dockerfile`, `docker-compose.yml` (dependencias) + `docker-compose.qa.yml` (aplicación, proxy `deploy/caddy/Caddyfile`), `scripts/qa.sh`; verificado: ambiente levantado, humo, persistencia (`documentacion/evidencias/entrega-8-verificacion.md`)	1, 8	✓ (local)
GEN-04	Azure + Terraform (aprovisionar, modificar, destruir)	`infrastructure/terraform/{bootstrap,azure}` (estado remoto protegido, VM, red, NSG, IP, discos). Ciclo aprovisionar, modificar y destruir ejecutado con proveedor simulado (`documentacion/evidencias/iac-sin-costo.md`). **Sin despliegue real**: el equipo decidió no usar recursos de pago	8, 12	◐
GEN-05	Ansible (SO, Docker, reverse proxy)	`infrastructure/ansible` (roles `base`, `docker`, `medicore`; proxy Caddy en el Compose). Aplicado sin fallas sobre un Ubuntu 24.04 desechable, con la aplicación funcionando (`documentacion/evidencias/iac-sin-costo.md`). **Sin servidor en Azure**	8, 12	◐
GEN-06	API REST documentada (OpenAPI)	`AddOpenApi` + `/openapi/v1.json` en Development (sin esquema Bearer ni comentarios aún)	1	◐
GEN-07	SonarQube + Quality Gate	Puerta "MediCore" aprobada; rechazó código nuevo en las corridas 2, 7 y 8, y aprobó la 9; `reports/sonarqube/`	11	✓
GEN-08	Fortify, Dependency-Check, npm audit	Dependency-Check, npm audit y NuGet ejecutados. **Fortify sin ejecutar: falta licencia** (`scripts/sec-fortify.sh`)	11	◐
GEN-09	OWASP ZAP sobre QA	Web y API con 5 roles: 0 alertas altas, 40 de 45 operaciones alcanzadas; `reports/zap/`	11	✓
GEN-10	Cypress (smoke, crítico, regresión, negativo)	`tests/cypress/e2e/*.cy.js` (10 specs; ver fila superior y `tests/cypress/README.md`)	10	◐

ID	Requisito	Implementación	Entrega	Estado
GEN-11	Maestro (5–8 flujos)	`tests/maestro/*.yaml` (8 flujos), `scripts/maestro_run.py` — 8/8 con el código final en emulador, contra QA	7, 10	✓
GEN-12	k6/JMeter: Load, Stress, Spike con umbrales previos	`tests/k6/medicore.js`, `documentacion/07-rendimiento.md`, `reports/k6/`; umbrales cumplidos, solo en equipo local	13	✓
GEN-13	Usabilidad (≥5 participantes) + Lighthouse/axe	axe: 14 pantallas, 0 infracciones. **Usabilidad definida en `08-usabilidad.md`, sin aplicar a personas**	13	◐
GEN-14	Registro de defectos con evidencia	`documentacion/12-registro-de-defectos.md`, plantilla `.github/ISSUE_TEMPLATE/defecto.yml`. 26 Issues creados en GitHub con `scripts/crear-issues.py` (23 cerrados y 3 abiertos, una vez cerrado H8-08)	continuo	◐
GEN-15	Plan de pruebas (20 secciones), riesgos C/I/D, casos, informe GO / NO-GO	`10-plan-de-pruebas.md`, `09-matriz-de-riesgos.md`, `11-casos-de-prueba.md`, `13-informe-final.md`	14	✓
GEN-16	Ambiente reproducible desde código y documentación	README + `documentacion/06-infraestructura.md` + `scripts/qa.sh` + Compose + migraciones por contenedor; reproducido localmente desde cero. Falta reproducirlo en Azure	1, 8, 12	◐
GEN-18	Respaldos y restauración (R-16, H8-08; requisito de continuidad)	`deploy/respaldos/respaldo.sh`, `scripts/simulacro-respaldo.sh`, `documentacion/evidencias/respaldos.md`; 14 tablas idénticas tras pérdida total	14	✓
GEN-17	Insomnia: colección +/−/authz	`tests/insomnia/medicore.insomnia.json`, `scripts/api-tests.sh`; 61 solicitudes, 65/65	10	✓